

Ivy Pulse — Sent and Received Message Models

Application version 1.0.0 · Entity and mesh delivery review · September 22, 2026

This document covers the general data formats exchanged by Ivy Pulse and its host SDK. It does not enumerate individual fault conditions or checklist items. TypeScript-style notation describes the JSON shapes: ? means a property may be omitted, null is an explicit JSON null, and date strings are UTC ISO 8601 unless noted otherwise.

1. Exchange overview

Model	Sent	Received
PMCS report	JSON string broadcast to peers; also stored in a Lattice entity's description	Parsed from an incoming peer payload or Lattice entity description
Report withdrawal	Small JSON peer payload; inactive entity update on Lattice	Peer payloads and inactive Lattice entities record a durable local withdrawal
Peer-message envelope	Added/managed by the host	Supplies sender ID, callsign, payload, and receipt time
Delivery result	Produced by the host SDK	Returned to Ivy Pulse after a peer broadcast

The extension has two application payload types: `ivy_pulse.report` and `ivy_pulse.deletion`. It uses the same report body on both transports. Host routing, radio framing, and host-internal conversion to Lattice protocols are outside these application models.

2. PMCS report payload

```
type Phase = "BEFORE" | "DURING" | "AFTER";
type Severity = "RED_X" | "CIRCLE_X" | "DASH";
type VehicleType = "STRYKER" | "JLTV";
type ISO8601UTC = string;
```

```
interface PmcsReportPayload {
  type: "ivy_pulse.report";
  entityId: string;          // Stable inspection/report UUID
  bumperNumber: string;
  vehicleType: VehicleType;
  operator: string;         // Signer display name or "UNVERIFIED"
  uic: string;
  phases: Phase[];          // Completed phases included in this report
  faults: PmcsFaultPayload[];
  signature?: SignaturePayload;
  latitude: number;
  longitude: number;
  timestamp: ISO8601UTC;    // Original report creation time
}
```

`entityId` links the peer report, Lattice entity, and queued retry. A new PMCS receives a new ID even when it uses the same vehicle. Bumper number and UIC are trimmed and uppercased for new inspections.

`faults` can be empty. Each new PMCS selects BEFORE, DURING, or AFTER and can be submitted as soon as that checklist is finished and signed. `phases` contains that inspection type; it does not represent progress toward a three-phase lifecycle. The list format still supports older reports and drafts containing multiple completed types. Only completed-checklist fault records are assembled into the submitted report. Serviceable answers and unanswered checks are not included as individual records.

The current app uses 0 for each missing coordinate. There is no separate location-availability field. Local database row IDs, read/unread flags, `isOutgoing`, delivery status, and `fromCallsign` are not part of this payload.

3. Fault object and optional description

```
interface PmcsFaultPayload {
```

```

itemId: string;           // Catalog check identifier
phase: Phase;
category: string;         // System or inspection station
subcategory: string;      // Component/check name
description: string;       // Catalog inspection instruction
condition: string;        // Selected fault condition
severity: Severity;
note: string | null;      // Optional operator description
recordedAt: ISO8601UTC;
}

```

The operator's optional 155-character description is note. The separate description field contains the catalog's inspection instruction. New note input is trimmed; blank input becomes null. The entry limit is 155 user-perceived Unicode characters. The note travels inside the report on both transports, including retries; it is not a separate message.

A check marked serviceable produces no fault object. The normal inspection flow retains one selected condition per check. Report status and fault counts are calculated from the fault list rather than transmitted as separate peer-payload fields.

4. Signature and identity models

```

type SignaturePayload =
  | {
    verified: true;
    identity: CacIdentityPayload;
    signedAt: ISO8601UTC;
  }
  | {
    verified: false;
    blockedBy: string;           // CAC rejection enum name
    attestedBy?: TypedIdentityPayload;
    signedAt: ISO8601UTC;
  };

interface CacIdentityPayload {
  edipi: string;               // DoD ID, stored as text
  firstName: string;
  lastName: string;
  middleInitial?: string;
  rank?: string;
  branchCode?: string;
  categoryCode?: string;
  cardExpiresOn?: ISO8601UTC;
  cardInstance?: string;
  verifiedAt: ISO8601UTC;
}

interface TypedIdentityPayload {
  edipi: string;
  firstName: string;
  lastName: string;
}

```

Signature case	Serialized content
Accepted CAC identity	verified: true, identity, and signedAt
Typed fallback	verified: false, blockedBy, attestedBy, and signedAt
Unverified override without a name	verified: false, blockedBy, and signedAt; operator is UNVERIFIED
Legacy report without a signature	Entire signature property omitted

Optional CAC identity fields are omitted when empty or unavailable. For the current verified submit path, signedAt uses the identity's verifiedAt; for an unverified override it uses the submission time. These timestamps remain unchanged during retries.

blockedBy uses an enum name such as noCamera, cancelled, or expired, rather than a full UI message. The current enum also supports noCodeFound, codeUnreadable, cardTooSmall, notACac, wrongSideOfCard, legacySsnCard, and cameraTimedOut.

“Verified” represents an accepted scanned identity in this app. The payload contains no cryptographic signature, certificate, CAC image, raw barcode, date of birth, or SSN.

5. Peer transport envelope

Ivy Pulse sends `JSON.stringify(reportPayload)` or `JSON.stringify(deletionPayload)` through the SDK's `messaging.broadcast(payload)` method, which targets all known peers. UIC filtering in the Reports screen does not filter broadcast recipients.

The web SDK passes this argument to the host bridge:

```
interface BroadcastBridgeArgument {
  payload: string; // JSON-encoded application payload, not a nested object
}
```

Incoming peer messages arrive at Ivy Pulse in the SDK envelope below. `receivedAt` is an integer number of milliseconds since the Unix epoch in the SDK JSON; the Dart SDK converts it to a `DateTime`.

```
interface IncomingMessageJson {
  id: string; // Host message ID, not report entityId
  fromPeerId: string;
  fromCallsign: string;
  payload: string; // Parse this to obtain report/deletion payload
  receivedAt: number; // Unix epoch milliseconds
}
```

The report decoder uses the envelope's `callsign` for local display, falling back to `Mesh` when empty. A report received through Lattice is locally labeled `Lattice`. Neither display label is injected into the PMCS report body.

6. Lattice entity model

For a normal PMCS publication, the extension builds the SDK entity below and calls `entities.upsertEntity(entity)`. This is the SDK JSON shape at the extension/host boundary, not a specification of the host's underlying Lattice wire protocol.

```
type ReportStatus = "FMC" | "FMC (DASH)" | "LIMITED" | "NMC";

interface PmcsLatticeEntityJson {
  id: string; // Same as report entityId
  name: string; // "<bumperNumber> - <status>"
  lat: number;
  lon: number;
  disposition: "friendly" | "hostile";
  shapeType: "point";
  description: string; // JSON-encoded PmcsReportPayload
  environment: "land";
  ontology: {
    platformType: "Stryker" | "JLTV";
    specificType: ReportStatus;
    template: "TEMPLATE_ASSET";
  };
  provenance: {
    integrationName: "ivy_pulse";
    dataType: "PMCS_REPORT";
    sourceUpdateTime: ISO8601UTC;
    sourceId: string;
  };
  status: {
    platformActivity: "PMCS";
    role: ReportStatus;
  };
  alternateIds: {
    id: string;
    type: "ALT_ID_TYPE_ASSET_ID";
  }[];
  createdTime: ISO8601UTC;
  expiryTime: ISO8601UTC;
  isLive: boolean;
  extra: {
    signedBy: string;
    signatureVerified: boolean;
    signatureMethod: "cac" | "typed" | "none";
    dodId?: string;
    signedAt?: ISO8601UTC;
  };
}
```

```
};
}
```

Signer summary fields are nested inside extra in SDK JSON. The full signature also remains inside the report encoded in description. `dodId` is present for scanned or typed identities. `signedAt` is present when a signature exists.

Fault summary	Computed status	Entity disposition
No faults	FMC	friendly
DASH faults only	FMC (DASH)	friendly
CIRCLE_X present, no RED_X	LIMITED	friendly
Any RED_X	NMC	hostile

These are the current app's entity metadata choices. New publications use `isLive: true`, with expiry one day after publication. Rebuilding an entity for retry or repair refreshes its entity publication timestamps and expiry; the embedded report retains its original timestamp and signature times.

On receive, Ivy Pulse selects entities whose provenance is `ivy_pulse / PMCS_REPORT`, records entities with `isLive: false` as withdrawals, and parses live entities' description JSON into the report model. The report ID in the description must match the entity ID. Other optional SDK entity fields can exist on host entities, but normal PMCS publication does not set them.

One legacy recovery case uses a different wrapper: if a queued report cannot be decoded, the app publishes `name: "PMCS <entityId>", platformType: "Vehicle", specificType: "PMCS", friendly disposition, raw stored payload as description, and the queue's coordinates. That fallback has no signer extra object and no status.role; the normal report-body guarantee does not apply to its description.`

7. Withdrawal model

Peer withdrawal uses this complete application payload:

```
interface PmcsDeletionPayload {
  type: "ivy_pulse.deletion";
  entityId: string;
}
```

There are no operator, timestamp, vehicle, or reason fields. On receipt, the entity ID identifies the matching local report to remove.

Lattice withdrawal uses an upsert of the existing entity with `isLive` changed to `false` and `expiryTime` set to one day after withdrawal so reconnecting clients can discover it. Other existing entity fields are retained. This is a tombstone update, not an SDK delete call. If the entity is already absent, there is no Lattice upsert.

Only withdrawing an outgoing report triggers these sends. Deleting someone else's received report removes the local copy only. Both withdrawal retries and the local withdrawal record are committed together before sending. Failed withdrawals retry automatically on their respective transports, including after restarting the app. An in-flight report publication finishes before its withdrawal is sent; older queued reports are discarded without republishing. Local withdrawal records also prevent delayed mesh messages or stale sync results from recreating the report.

8. Delivery-result model and retries

The peer SDK returns this result to the sender. It is local SDK feedback rather than an Ivy Pulse application message broadcast to peers.

```
interface DeliveryReport {
  results: DeliveryResult[];
}
interface DeliveryResult {
  peerId: string;
```

```

    success: boolean;
    error: string | null;
}

```

Ivy Pulse treats a peer broadcast as successful when at least one result succeeds. It does not interpret that as confirmation from every peer. Lattice publication is treated as successful only when `getEntity(id)` returns the expected ID and report description with Ivy Pulse provenance, and the returned entity is not inactive. This confirms the host's stored copy, not reception by every mesh device.

Failed report sends are queued independently by transport. Peer retries reuse the saved JSON string. Lattice retries decode the saved report and rebuild its entity. Normal report retry probes run on a nominal 45-second timer; remote synchronization runs on startup and then every five minutes and can repair missing local reports on Lattice. Each transport is retried independently, so a stalled Lattice prompt or send cannot block mesh retries. A failed host listing aborts repair instead of being interpreted as an empty host. Neither produces a new application message type.

Repeated deliveries are identified by `entityId`, which is protected against duplicate local report rows. Concurrent refresh/send work is coordinated, but network delivery is not guaranteed to be exactly once. Sending, Queued, and Sent are local UI states, not fields appended to the PMCS payload.

9. Receive compatibility

The models above describe current output. The receiver also supports older input:

- Peer payloads must have a recognized type. A peer report requires a nonempty `entityId`, a string bumper number, a recognized vehicle type, and numeric latitude/longitude. Unusable payloads are ignored.
- A missing `uic` can fall back to the older `unit` field. A missing signature remains unverified. Unknown extra properties are not included in the reconstructed domain report.
- Unknown phase-list entries are skipped. Missing phase/fault lists default to empty lists. Missing or invalid report timestamps default to receive-time decoding; these are compatibility defaults, not current output conventions.
- A valid identity block establishes the decoder's verified signature state; the `verified` boolean by itself is insufficient. An unknown unverified reason falls back to `noCodeFound` when a signature can otherwise be reconstructed.
- Peer deletion requires the exact deletion type and a nonempty string entity ID. Lattice imports use provenance/live filtering and the description body rather than the peer-message envelope.

There is no explicit schema-version field in the current application payload.

10. Compact JSON example

This is an actual serializer-produced report shape with synthetic identity/vehicle data. The single fault illustrates structure only.

```

{
  "type": "ivy_pulse.report",
  "entityId": "11111111-1111-4111-8111-111111111111",
  "bumperNumber": "A-11",
  "vehicleType": "STRYKER",
  "operator": "UNVERIFIED",
  "uic": "DEM001",
  "phases": [
    "BEFORE"
  ],
  "faults": [
    {
      "itemId": "B-EXT-01",
      "phase": "BEFORE",
      "category": "EXTERIOR / HULL",
      "subcategory": "Hull & Body Panels",
      "description": "Inspect hull for cracks, dents, holes, or battle damage",
      "condition": "Cracks Found",
      "severity": "DASH",
      "note": null,
      "recordedAt": "2026-09-21T13:57:00.000Z"
    }
  ]
}

```

```
    }  
  ],  
  "signature": {  
    "verified": false,  
    "blockedBy": "cancelled",  
    "signedAt": "2026-09-21T14:00:00.000Z"  
  },  
  "latitude": 38.75,  
  "longitude": -104.79,  
  "timestamp": "2026-09-21T14:00:00.000Z"  
}
```

Implementation references

- Report/deletion codec
- Fault model, signature model
- Lattice mapper, Lattice adapter
- Peer broadcaster, Lattice receiver
- SDK messaging models
- SDK entity JSON, web bridge